Augmented Reality with ArUco Markers

Hochschule Ravensburg-Weingarten

Jonas Alber
Master Informatik
15444427
jonas.alber@hs-weingarten.de

Tim Schweitzer
Master Informatik
29844429
tim.schweitzer@hs-weingarten.de

January 8, 2025

Disclaimer

The content of this document was created by the authors mentioned above. ChatGPT from OpenAI was used to improve readability. However, all content has been checked by the authors and adapted where necessary.

List of abbreviations

RWU Hochschule Ravensburg Weingarten

KITTI Karlsruhe Institute of Technology and Toyota Technological Institute

YOLO You only look once

Bbox Bounding Box

IoU Intersection over Union

CNN Convolutional Neural Networks

1 Task definition

This work was created as part of the Computer Vision course at Hochschule Ravensburg Weingarten (RWU) as a project assignment. The aim of the project is to use the You only look once (YOLO) model for detection cars inside provided images. Additionally the distance from the camera to the car is calculated. The images are provided by the Karlsruhe Institute of Technology and Toyota Technological Institute (KITTI) dataset. The project is divided into two main tasks:
This image augmentation is applied to a given data set, whereby the results achieved are then to be transferred to self-created images.

2 Setup

2.1 Neuronal Network

YOLO is a Convolutional Neural Network (CNN) that processes images through a single pass, making it efficient for detecting multiple objects simultaneously in an image. YOLOv11 was chosen for its fast and accurate object detection capabilities.

2.2 Preprocessing

To utilize the KITTI dataset for our object detection and depth estimation task, we prepared the data by resizing images and converting the annotation format. This step ensures compatibility with the YOLOv11 object detection framework. The preprocessing script, implemented in Python, performs the following tasks:

- Annotation Conversion.
- Image Resizing.

The script processes the dataset once, converting and saving the images and annotations in the required format.

2.2.1 Annotation Conversion

The images used define the Bounding Box (Bbox) using the KITTI standard. This definition is not compatible with the Bbox definition of YOLO, so they need to be converted. The KITTI defines the Bbox with the following parameters:

- `x_min, y_min`: Upper left corner coordinates of the image.
- `x_max, y_max`: Lower right corner coordinates of the image.

so that the resulting Bbox looks like:

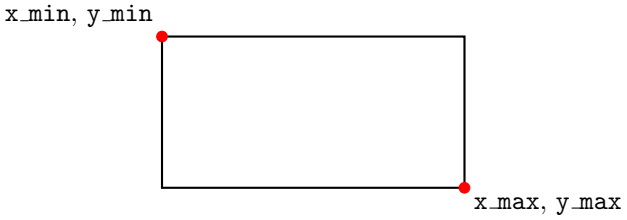


Figure 1: Example of a bounding box in KITTI format.

In contrast, the YOLO format specifies each bounding box as:

- `class_id`: Numeric identifier for the object class.
- `x_center, y_center`: Normalized coordinates of the bounding box center.
- `width, height`: Normalized dimensions of the bounding box.

KITTI does not support the class identifier, so we omitted this field in the conversion. Therefore, car is identified as 0. To convert the bounding box coordinates, we use the following formulas:

$$\text{img\_width} = x_{\max} - x_{\min}, \quad \text{img\_height} = y_{\max} - y_{\min}$$

$$x_{\text{center}} = \frac{x_{\min} + x_{\max}}{2 \cdot \text{img\_width}}, \quad y_{\text{center}} = \frac{y_{\min} + y_{\max}}{2 \cdot \text{img\_height}}$$

$$\text{width} = \frac{x_{\max} - x_{\min}}{\text{img\_width}}, \quad \text{height} = \frac{y_{\max} - y_{\min}}{\text{img\_height}}$$

The resulting bounding box in YOLO format is shown in Figure 2.

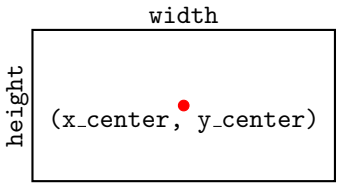


Figure 2: Example of a bounding box in YOLO format.

2.2.2 Image Resizing

To standardize the input dimensions for the object detector, all images were resized to 1248×512 pixels using OpenCV. While the original images are 1242×375, resizing to 1248×512 ensures compatibility with YOLOv11's input requirements, without significant loss of detail or increase in computational cost.

3 Training

3.1 Training Configuration

We split the dataset into 80% for training and 20% for validation, the splitting is done via random selection, resulting in 16 images for training and 4 images for validation. This 80:20 split is a common practice in training Convolutional Neural Networks (CNN) [1] as it provides a good balance between having enough data to train the model and enough data to validate its performance.

The training configuration included the following key hyperparameters: The model was trained for 100 epochs. An epoch refers to one complete pass through the entire training dataset. Training for multiple epochs helps the model to learn and generalize better. A batch size of 32 was used. This means that 32 samples from the training dataset were processed before the model's internal parameters were updated. A larger batch size can lead to more stable gradient updates. Data augmentation was enabled (set to True). Data augmentation involves applying random transformations to the training data, such as rotations, translations, and flips, to artificially increase the diversity of the training dataset and improve the model's robustness.

3.2 Training Process

The trained YOLOv11 model was evaluated on the validation set to measure detection accuracy. Predictions included bounding boxes, class labels, and confidence scores.Intersection over Union (IoU) was used to assess overlap between predicted and ground-truth boxes.

A precision-confidence curve 3 was generated to show how precision improves as low-confidence predictions are filtered out, providing insight into the model's reliability. The next sections present detailed metrics and results for a comprehensive performance analysis.

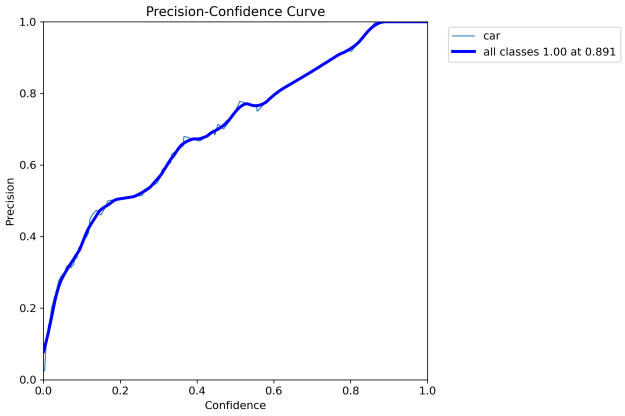


Figure 3: Precision Confidence Curve from the trained Model from the model validation

4 IoU Calculation

4.1 Function Description

The function takes two bounding boxes as input, defined as:

$$\text{groundTruthBox} = [x_1, y_1, x_2, y_2],$$

$$\text{predictedBox} = [x_{1g}, y_{1g}, x_{2g}, y_{2g}]$$

where:

- (x_1, y_1) and (x_2, y_2): Top-left and bottom-right coordinates of the first box.

- (x_{1g}, y_{1g}) and (x_{2g}, y_{2g}): Top-left and bottom-right coordinates of the second box.

4.2 Steps in IoU Calculation

1. **Intersection Box Calculation**: The intersection box is determined by finding the maximum of the top-left coordinates and the minimum of the bottom-right coordinates:

$$x_i^1 = \max(x_1, x_{1g}), \quad y_i^1 = \max(y_1, y_{1g})$$

$$x_i^2 = \min(x_2, x_{2g}), \quad y_i^2 = \min(y_2, y_{2g})$$

The area of the intersection box is then computed as:

$$\text{inter\_area} = \max(0, x_i^2 - x_i^1) \cdot \max(0, y_i^2 - y_i^1)$$

2. **Areas of the Individual Boxes**: The areas of the two bounding boxes are calculated as:

$$\text{box1\_area} = (x_2 - x_1) \cdot (y_2 - y_1),$$

$$\text{box2\_area} = (x_{2g} - x_{1g}) \cdot (y_{2g} - y_{1g})$$

3. **Union Area Calculation**: The union area is computed as the sum of the individual box areas minus the intersection area:

$$\text{union\_area} = \text{box1\_area} + \text{box2\_area} - \text{inter\_area}$$

4. **IoU Calculation**: Finally, the IoU is obtained by dividing the intersection area by the union area:

$$\text{IoU} = \frac{\text{inter\_area}}{\text{union\_area}}$$

5. **Matching**: To match the predicted Bounding with the corresponding Groundtruth each prediction calculates the IoU with every Groundtruthbox and only keeps the best result with the largest overlaping area.

5 Horizontal Distance Calculation

The function computes the horizontal distance from the camera to the point where the ray, originating from the lower center of a bounding box in the image, intersects with the ground plane. This process uses the intrinsic matrix of the camera, the height of the camera, and the assumption that the images are all on a flat plane.

5.1 Bounding Box Center (Bottom-Center)

The bounding box is represented by its corner coordinates: $(x_{\min}, y_{\min})$ for the upper left corner and $(x_{\max}, y_{\max})$ for the lower right corner. To find the center of the bounding box, we compute the average of the x-coordinates of the box corners:

$$x_{\text{center}} = \frac{x_{\min} + x_{\max}}{2}$$

The y-coordinate is taken from the bottom of the box, so:

$$y_{\text{center}} = y_{\max}$$

This step locates the center of the bounding box at the bottom-center position, which is used for further calculations.

5.2 Inverse of Intrinsic Matrix

The camera's intrinsic matrix K relates pixel coordinates in the image to the camera's coordinate system. To convert pixel coordinates to camera coordinates, we need the inverse of the intrinsic matrix. This is computed as follows:

$$K_{\text{inv}} = \text{np.linalg.inv}(K)$$

The inverse matrix K_{inv} allows us to map the pixel coordinates to normalized camera coordinates.

5.3 Convert Pixel Coordinates to Camera Coordinates

Using the inverse intrinsic matrix, we can convert the pixel coordinates of the bounding box center to camera coordinates. The center's pixel coordinates $(x_{\text{center}}, y_{\text{center}})$ are used to form a direction vector, which points from the camera center toward the center of the bounding box:

$$\text{ray\_direction} = K_{\text{inv}} \cdot \begin{bmatrix} x_{\text{center}} \\ y_{\text{center}} \\ 1 \end{bmatrix}$$

We then normalize this direction vector to ensure it has a unit length:

$$\text{ray\_direction} = \frac{\text{ray\_direction}}{\|\text{ray\_direction}\|}$$

The components of the normalized ray direction are denoted as r_x, r_y, and r_z:

$$\mathbf{r} = \begin{bmatrix} r_x \\ r_y \\ r_z \end{bmatrix} = \text{ray\_direction}$$

5.4 Calculate Scaling Factor (t) for Ground Plane Intersection

To compute the intersection of the ray with the ground plane, we calculate a scaling factor t, which scales the direction vector such that the ray intersects the ground. The scaling factor is calculated by dividing the camera height by the value of r_y, the vertical component of the direction vector:

$$0 = t \cdot r_y - \text{camera\_height}$$

$$t = \frac{\text{camera\_height}}{r_y}$$

5.5 Compute Intersection Point in Camera Coordinates

Once the scaling factor t is determined, we can compute the intersection point in camera coordinates by scaling the direction vector:

$$\text{intersection\_camera} = t \cdot \text{ray\_direction}$$

The ray length, which is the distance from the camera to the intersection point, is given by the magnitude of the intersection point:

$$\text{rayLength} = \|\text{intersection\_camera}\|$$

5.6 Compute Horizontal Distance

Finally, the horizontal distance from the ground plane is calculated by projecting the intersection point onto the ground. This is done by calculating the distance between the camera and the intersection point. The horizontal distance is given by:

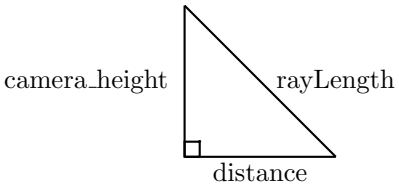


Figure 4: Right Triangle Representation of the Pythagorean Theorem for Camera Height, Distance, and Ray

$$\text{horizontal\_distance} = \sqrt{\text{rayLength}^2 - \text{camera\_height}^2}$$

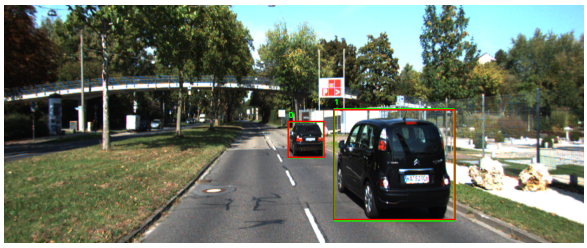
This value represents the horizontal distance from the camera to the point where the ray intersects the ground. Since this point is in the lower center of the bounding box, this is the distance from the camera to the car.

6 IoU and Distance Calculation on Images

6.1 Setup of the images in the document

The formulas for IoU and Horizontal Distance Calculation defined in the previous chapters are now applied to the provided images. The results of the application are as follows. In Figure 6.2, the result of applying the formulas to image 006374 is shown. The results of applying the formulas to the other images can be found in the Results section. Below each image, there is a table listing the detected objects with their ID from the image. The first column indicates the detected class and the model's confidence in the detection. Next, the IoU for each bounding box is provided, followed by the precision and recall of the model for the corresponding image. The last two columns show the calculated distance values using the formulas from Compute Horizontal Distance as CalcDist and the ground truth distance provided with the image annotations.

6.2 Output Image 006374



ID	Label:Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.99	0.95	1.00	1.00	7.69	18.22
1	car: 0.90	0.97	1.00	1.00	4.29	6.93

7 Summary

This project focuses on object detection and distance estimation using the YOLOv11 model on the KITTI dataset. Preprocessing included resizing images and converting KITTI-format annotations into YOLO-compatible bounding boxes to ensure compatibility with the model.

The dataset was split 80:20 for training and validation. Key training parameters were 100 epochs, a batch size of 32, and data augmentation to improve robustness. Evaluation relied on the IoU to measure prediction accuracy, with the approach effectively distinguishing true positives from false positives. The IoU-based detection precision performed well, indicating reliable localization of objects.

Distance estimation calculated horizontal distances from the camera to objects by projecting bounding box centers to the ground plane using the camera's intrinsic matrix and height. While this method worked well for shorter distances, accuracy degraded significantly beyond 15 meters. This highlights either a limitation in using single-image geometry for long-range depth estimation. Or the implementation has an undiscovered bug which degraded the result massively.

Overall, the project successfully integrated object detection and distance estimation but showed the need for enhanced techniques, such as stereo vision or sensor fusion, to improve long-range performance.

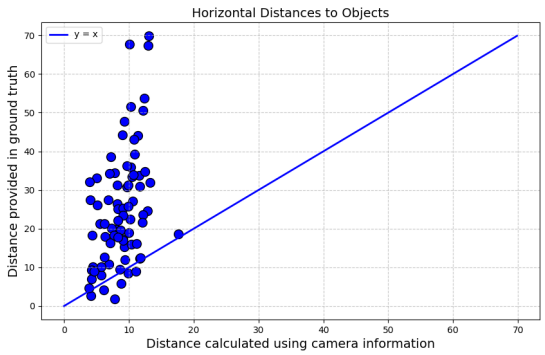


Figure 5: Distance comparison between calculated und Groundtruth

References

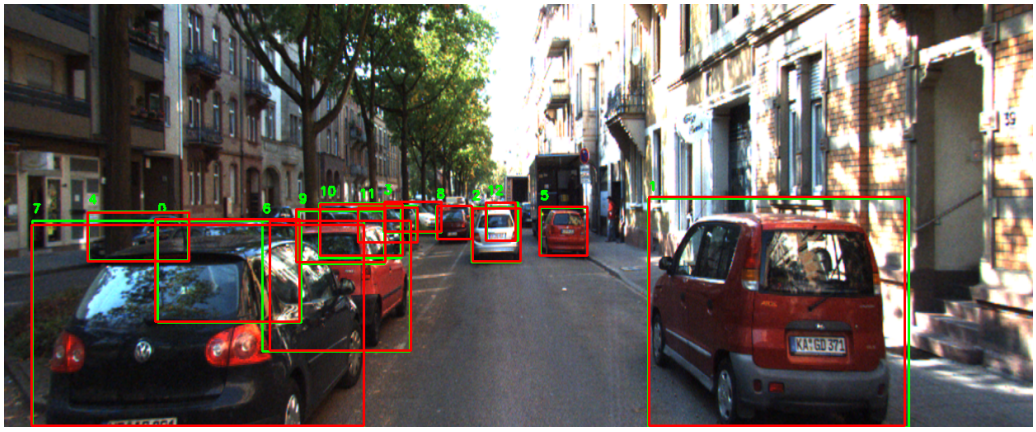
[1] Afshin Gholamy, Vladik Kreinovich, Olga Kosheleva. *Why 70/30 or 80/20 Relation Between Training and Testing Sets: A Pedagogical Explanation.* Technical Report: UTEP-CS-18-09.

A Results

A.1 output image006206

ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.43	0.97	1.00	1.00	11.37	44.10

A.2 output image006211



ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.95	0.95	1.00	1.00	6.19	21.28
1	car: 0.94	0.97	1.00	1.00	3.86	4.61
2	car: 0.94	0.95	1.00	1.00	8.56	9.42
3	car: 0.93	0.96	1.00	1.00	11.69	12.35
4	car: 0.93	0.95	1.00	1.00	10.21	22.52
5	car: 0.93	0.97	1.00	1.00	9.12	25.35
6	car: 0.90	0.94	1.00	1.00	5.01	33.02
7	car: 0.89	0.97	1.00	1.00	3.96	32.10
8	car: 0.89	0.95	1.00	1.00	10.75	43.07
9	car: 0.83	0.91	1.00	1.00	8.75	5.77
10	car: 0.77	0.91	1.00	1.00	9.14	23.39
11	car: 0.75	0.93	1.00	1.00	10.54	27.16
12	car: 0.38	0.90	1.00	1.00	10.52	33.41

A.3 output image006329

ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.91	0.98	1.00	1.00	4.29	9.39
1	car: 0.88	0.98	1.00	1.00	6.12	4.11
2	car: 0.86	0.94	1.00	1.00	4.67	8.90

A.4 output image006315

ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.97	0.97	1.00	1.00	7.15	16.30
1	car: 0.39	1.00	1.00	1.00	12.38	53.73

A.5 output image006310

ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.96	0.97	1.00	1.00	6.78	27.48
1	car: 0.95	0.95	1.00	1.00	9.91	25.79
2	car: 0.94	0.95	1.00	1.00	9.96	18.92
3	car: 0.92	0.99	1.00	1.00	10.35	15.99
4	car: 0.87	1.00	1.00	1.00	17.59	18.58
5	car: 0.75	0.94	1.00	1.00	7.17	38.61
6	car: 0.71	0.88	1.00	1.00	12.95	67.33

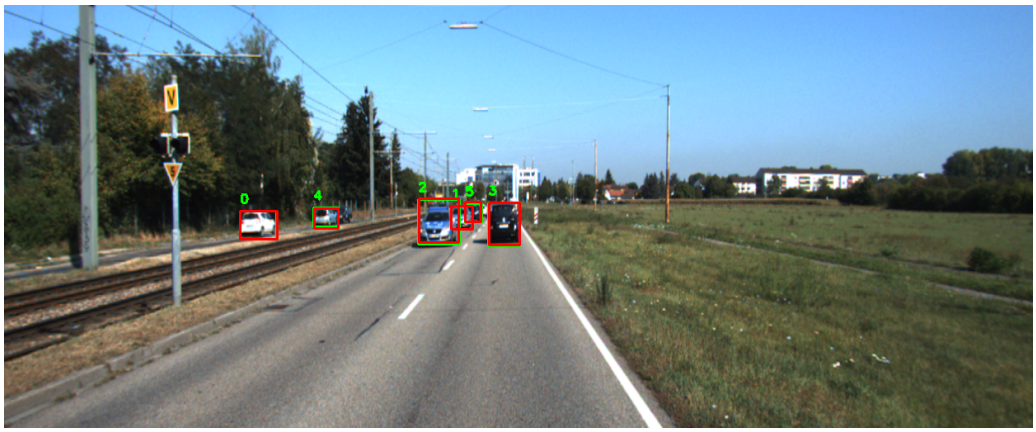
A.6 output image006098

ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.95	0.95	1.00	1.00	11.80	12.53
1	car: 0.94	0.96	1.00	1.00	8.28	17.84
2	car: 0.91	0.95	1.00	1.00	7.06	34.35
3	car: 0.82	0.95	1.00	1.00	12.43	34.72

A.7 output image006067

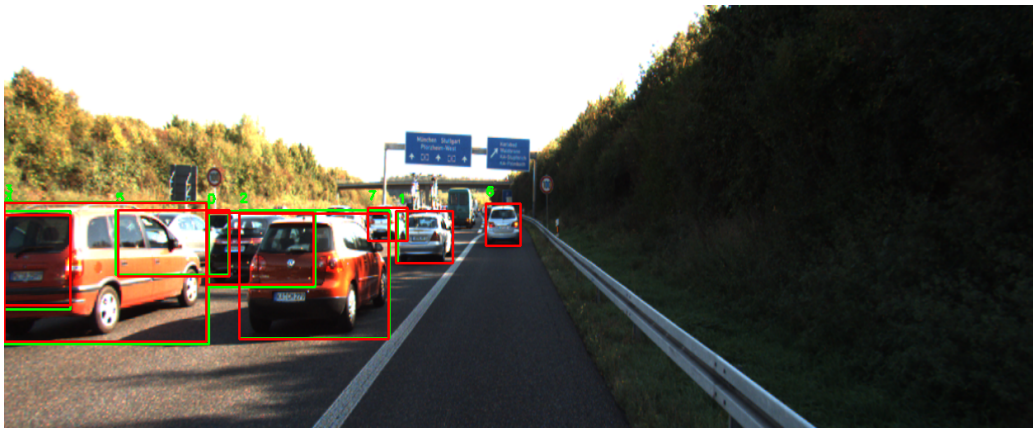
ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.91	0.91	1.00	1.00	12.82	24.56
1	car: 0.90	0.95	1.00	1.00	9.04	44.21
2	car: 0.90	0.97	1.00	1.00	10.66	33.90
3	car: 0.83	0.91	1.00	1.00	12.16	23.64
4	car: 0.54	0.95	1.00	1.00	9.32	47.77

A.8 output image006059



ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.91	0.98	1.00	1.00	11.71	30.90
1	car: 0.90	1.00	1.00	1.00	12.16	50.60
2	car: 0.90	0.91	1.00	1.00	10.32	51.56
3	car: 0.70	0.95	1.00	1.00	10.08	67.68
4	car: 0.61	0.85	1.00	1.00	13.08	69.87
5	car: 0.54	0.84	1.00	1.00	13.23	31.96

A.9 output image006312

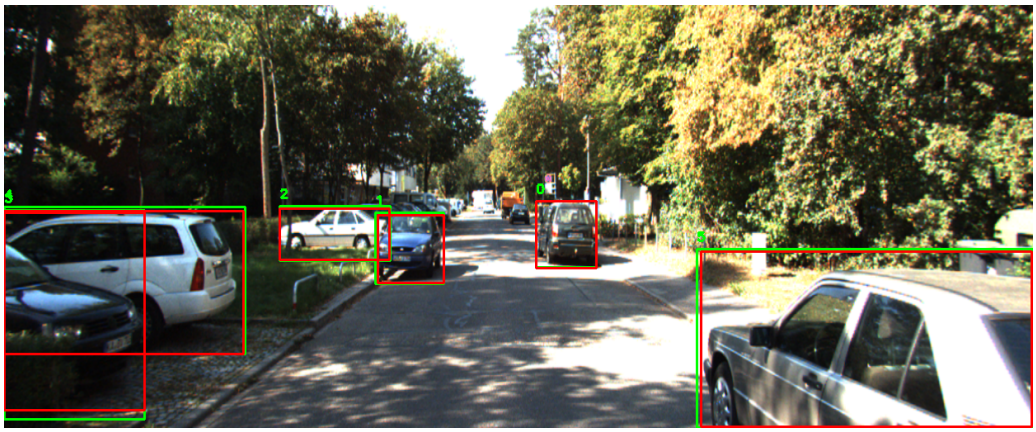


ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.96	0.93	1.00	1.00	8.23	31.22
1	car: 0.95	0.97	1.00	1.00	9.44	11.90
2	car: 0.94	0.96	1.00	1.00	5.72	10.14
3	car: 0.94	0.96	1.00	1.00	6.36	17.89
4	car: 0.92	0.92	1.00	1.00	8.27	22.10
5	car: 0.91	0.95	1.00	1.00	9.66	36.33
6	car: 0.87	1.00	1.00	1.00	11.21	16.12
7	car: 0.74	0.98	1.00	1.00	12.12	21.55

A.10 output image006121

ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist

A.11 output image006097

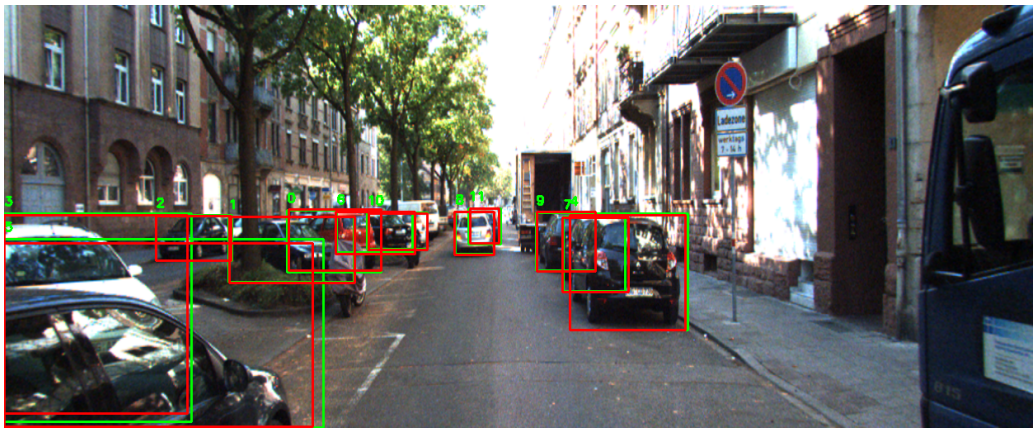


ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.98	0.99	1.00	1.00	8.30	25.11
1	car: 0.97	0.92	1.00	1.00	7.35	20.21
2	car: 0.96	0.93	1.00	1.00	9.10	16.90
3	car: 0.95	0.94	1.00	1.00	4.49	10.08
4	car: 0.92	0.97	1.00	1.00	5.68	7.94
5	car: 0.85	0.96	1.00	1.00	4.11	2.62

A.12 output image006130

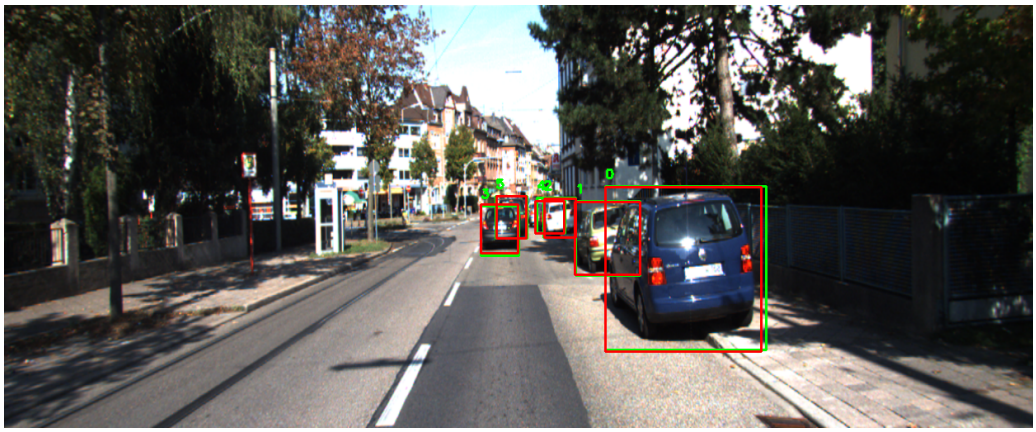
ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist

A.13 output image006291



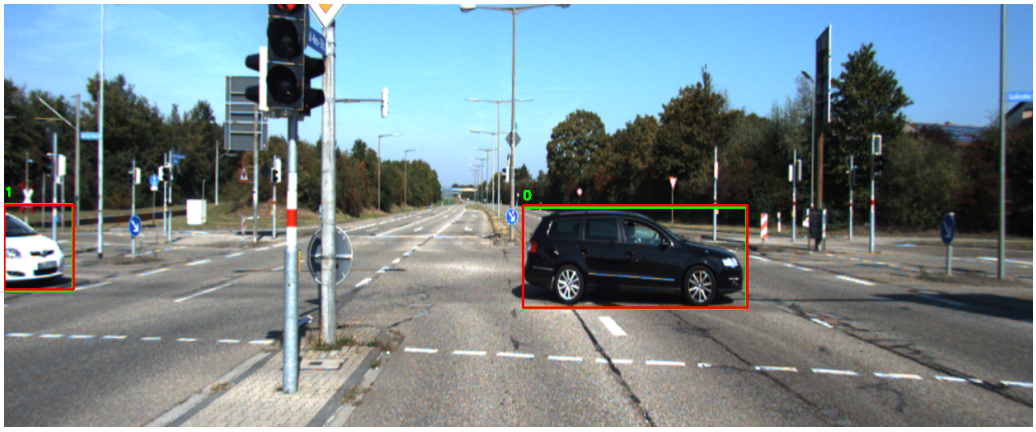
ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.97	0.94	1.00	1.00	8.26	26.52
1	car: 0.96	0.97	1.00	1.00	7.81	1.80
2	car: 0.95	0.97	1.00	1.00	9.88	8.43
3	car: 0.94	0.93	1.00	1.00	4.36	18.23
4	car: 0.94	0.97	1.00	1.00	5.52	21.23
5	car: 0.93	0.94	1.00	1.00	4.06	27.50
6	car: 0.93	0.92	1.00	1.00	9.67	30.79
7	car: 0.90	0.92	1.00	1.00	6.95	10.77
8	car: 0.90	0.96	1.00	1.00	9.30	15.32
9	car: 0.88	0.96	1.00	1.00	8.08	19.81
10	car: 0.75	0.93	1.00	1.00	9.91	31.32
11	car: 0.50	0.88	1.00	1.00	10.26	35.97

A.14 output image006253



ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.97	0.95	1.00	1.00	5.09	26.09
1	car: 0.93	0.97	1.00	1.00	7.83	34.38
2	car: 0.90	0.95	1.00	1.00	11.05	8.91
3	car: 0.79	0.90	1.00	1.00	9.15	17.29
4	car: 0.59	0.84	1.00	1.00	11.57	33.79
5	car: 0.58	0.95	1.00	1.00	10.91	39.23

A.15 output image006227



ID	Label: Confidence	IoU	Precision	Recall	CalcDist	GtDist
0	car: 0.93	0.96	1.00	1.00	6.24	12.67
1	car: 0.81	0.96	1.00	1.00	8.70	19.70